

Claude Code 및 Agentic Loops 분석 보고서

1.

<https://claude.com/blog/how-datadog-built-a-universal-machine-tool-f-or-claude-code>

1. 문서 개요

- 웹페이지 한 줄 핵심 정의: 데이터독(Datadog)이 클로드 코드(Claude Code)를 활용하여 소프트웨어 개발의 검증 문제를 해결하고, 에이전트 기반 개발을 정밀 제어하기 위해 자체 개발한 결정론적 커널 시스템인 '템퍼(Temper)' 구축 사례 분석 문서입니다.
- 작성 목적 및 주요 대상: AI 에이전트 도입에 따른 소프트웨어 개발 패러다임의 변화와 한계를 설명하고, 에이전트가 안전하고 반복 가능하게 복잡한 시스템을 구축할 수 있도록 지원하는 프레임워크 설계 패턴을 공유하기 위함입니다. 주요 대상은 엔지니어링 리더, AI 개발자, DevOps 전문가 및 엔터프라이즈 시스템 아키텍트입니다.

2. 핵심 주제 및 요약

- 데이터독 엔지니어 전체가 생성형 AI 코딩 도구를 활용 중이며, 그 중 클로드 코드가 약 3분의 2 이상을 차지할 정도로 높은 비중을 차지합니다.
- 코드를 생성하는 속도가 인간의 리뷰 및 검증 속도를 앞지르면서 발생한 '검증의 갭'을 해결하기 위해, 데이터독은 에이전트가 임의의 응용 프로그램 코드를 직접 작성하는 대신 명확한 명세서(Specification)를 작성하도록 전환했습니다.
- 이를 위해 개발된 커널 시스템 '템퍼(Temper)'는 에이전트가 생성한 명세를 4단계 레이어를 거쳐 검증하고 실행함으로써, 소프트웨어 개발 환경을 단순 코드 생성을 넘어 산업화 및 규격화된 공정으로 발전시키고 있습니다.

3. 상세 내용 분해 에이전트 도입과 개발 흐름의 변화

(The Flow Problem)

- 개발 역할의 전환: 과거에는 문제를 이해하고 코드를 작성 및 테스트하여 배포하는 일직선 형태의 작업이었으나, 클로드 코드와 같은 장기 실행 에이전트 도입 후 엔지니어의 역할이 직접 코딩하는 방식에서 작업의 형태와 도구, 성공 기준을 정의하는 관리 및 아키텍처 수립 역할로 상향 이동했습니다.

- 에이전트 확장의 한계: 에이전트가 스스로 도구를 만들고 응용 시스템을 커스텀하는 능력이 커질수록, 실행 결과와 인간 중심 도구 간의 격차가 벌어지며 검증과 통제가 점차 어려워지는 현상이 발생했습니다.

데이터독의 템퍼(Temper) 도달 과정 (The Road to Temper)

데이터독은 템퍼를 구축하기 전, 세 가지 주요 프로젝트를 진행하며 한계점을 파악하고 진화 과정을 거쳤습니다.

- **쿠리에 (Courier):** 2024 년에 구축한 분산 큐 시스템입니다. 전체를 수작업으로 구축하는 데 1 년이 걸렸으며, 컴포넌트 간의 상호작용을 관찰, 테스트, 검증하는 정밀한 형식 모델링 및 시뮬레이션의 중요성을 확인했습니다.
- **비츠이볼브 (BitsEvolve):** 2025 년 9 월에 구축된 닫힌 루프(Closed-loop) 진화적 최적화 하네스입니다. 모델 의회가 코드 변형을 생성하면 벤치마크와 라이브 관찰 가능성 피드백을 통해 생존 여부를 결정하는 시스템입니다. 이 과정에서 소프트웨어가 유기체처럼 자라날 수 있음을 확인했으나, 환경 피드백 루프의 속도가 병목이 됨을 파악했습니다.
- **헬릭스 (Helix):** 카프카(Kafka)와 비교 가능한 스트리밍 서비스로, 클로드 코드가 대부분의 시스템을 수일 만에 구축했습니다. 생성 속도는 대폭 향상되었으나, 프로덕션 환경으로 안전하게 배포하고 운영 단단함을 확보하는 검증 단계에서 다시 인간 중심 도구의 병목 현상이 발생했습니다.

범용 기계 공구 개념으로서의 템퍼 (Temper)

- 개념 정의: 제조업에서 엔진이나 정밀 부품을 만들기 위해 사용되는 정밀 지그, 고정구, 밀링 공구와 같이, 에이전트 시스템을 위한 정밀하고 반복 가능한 최적의 커널 프레임워크입니다.
- 동작 방식의 대전환: 템퍼는 에이전트에게 애플리케이션 코드 자체를 생성하도록 하지 않고, 제어 로직에 대한 정밀한 명세서(Specification)를 출력하도록 요구합니다.
- 컴파일과 검증의 분리: 명세 자체가 검증 대상이자 실행 아치팩트가 되므로, 검증된 내용과 실제 실행되는 시스템 간의 괴리(Drift)가 발생하지 않습니다. LLM 외부에서 별도의 결정론적 커널이 컴파일과 검증을 수행합니다. 템퍼 명세를 구성하는 **3** 가지 계약 (Contracts)
- 행동 계약 (Behavior): 시스템이 가져야 하는 상태, 상태 전환, 전제 조건 및 안전성 속성을 정의합니다.

- 데이터 계약 (**Data Contract**): 엔티티 타입, 속성, 지원 동작을 기계가 해석 가능한 형태로 게시하여 에이전트가 별도 문서 없이 전체 API 를 탐색하고 추론할 수 있도록 합니다.
- 인가 계약 (**Authorization**): 기본 거부(Default-deny) 정책을 기반으로 작동하며, 거부된 항목은 승인 대기 결정 항목으로 기록되어 인간이 승인 후 정책 엔진에 다이내믹하게 핫로드(Hot-load)할 수 있습니다.

템퍼의 4 단계 검증 레이어 (**Verification Cascade**)

에이전트가 생성한 명세는 커널에 로드되기 전 다음 4 가지 독립 레이어를 통과해야 하며, 소형 명세 기준 전체 검증 과정은 1 초 미만에 완료됩니다.

- 기호 추론 (**Symbolic Reasoning**): 모든 가드 조건이 충족 가능한지, 불변 조건(Invariant)이 귀납적으로 유지되는지 증명합니다.
- 전수 상태 탐색 (**Exhaustive State Exploration**): 도달 가능한 모든 상태를 방문하여 논리적 오작동을 검증합니다.
- 결정론적 시뮬레이션 (**Deterministic Simulation**): 패킷 낙하, 지연, 순서 변경, 크래시 등 난수 기반의 오류를 주입하여 실제 프로덕션 코드 경로를 실행하며, 동일한 시드값을 통해 오류를 100% 동일하게 재현합니다.
- 무작위 속성 테스트 (**Randomized Property Testing**): 약 1,000 개의 의사 난수 작업 시퀀스를 실행하고, 위반 사항이 발견되면 최소한의 반례로 축소하여 분석을 용이하게 합니다.

다크 팩토리(**Dark Factory**)와 헬릭스 운영

- 다크 팩토리에서의 역할: 가상 공장 현장에서 인간의 개입 없이 에이전트가 계속 작업하는 운영 구조를 의미합니다. 템퍼는 에이전트 제어 평면, Git/CI/배포 도구를 연결하는 도구 제작 레이어, 데이터 평면 주변의 제어 API 역할을 모두 수행합니다.
- 기존 **CRUD** 애플리케이션과의 차이: 기존 방식은 제어 로직이 라우트, DB 제약조건, 서비스 코드 등에 분산되어 상태 머신이 암묵적으로 존재했습니다. 반면 템퍼는 상태 머신을 명시적인 데이터(전환 테이블)로 분리하여 에이전트가 안전하게 동적으로 변경하고 CI 과정 없이 핫로드할 수 있도록 지원합니다.

4. 주요 시사점 및 결론

- 생성보다 검증에 투자: 에이전트 도입 시 진짜 병목은 코드 생성 속도가 아니라 생성된 결과물의 검증 속도와 안전성 확보에 있습니다.
 - 명세 기반의 개발 프레임워크 전환: 코드 형태의 스파게티 제어 흐름 대신 상태 머신을 명시적 데이터로 다루고, LLM 외부의 결정론적 커널에 검증을 맡기는 구조가 필요합니다.
 - 소프트웨어 개발의 진화: 정밀한 명세와 안전한 검증 환경이 구축되면, 소프트웨어는 단순한 코딩을 넘어 피드백과 환경 적응을 통해 지속적으로 육성되고 진화하는 유기체 형태로 발전할 수 있습니다.
-

2. <https://claude.com/blog/getting-started-with-loops>

1. 문서 개요

- 웹페이지 한 줄 핵심 정의: 앤트로픽(Anthropic)의 Claude Code 팀이 정의하는 '에이전트 루프(Agentic Loops)'의 개념과 유형별 활용법, 그리고 비저블 및 토큰 효율성을 유지하는 실전 공학(Loop Engineering) 가이드 문서입니다.
- 작성 목적 및 주요 대상: 단순히 단발성 프롬프트(Prompting)에 의존하는 것을 넘어 에이전트가 종료 조건에 도달할 때까지 작업 사이클을 반복하는 다양한 루프 패턴을 이해시키고, 개발자가 이를 시스템에 적용하도록 돕기 위함입니다. 주요 대상은 Claude Code 기반 소프트웨어를 개발하는 엔지니어 및 AI 에이전트 아키텍트입니다.

2. 핵심 주제 및 요약

- 에이전트 루프는 에이전트가 특정 중단 조건이 충족될 때까지 작업 사이클을 스스로 반복 수행하는 메커니즘을 의미합니다.
- 루프는 실행 방식과 종료 조건에 따라 턴 기반(Turn-based), 목표 기반(Goal-based), 시간 기반(Time-based), 능동형(Proactive)의 4 가지 유형으로 나뉩니다.
- 복잡한 루프 구축에 앞서 가장 단순한 형태부터 도입하고, SKILL.md 검증 도구 및 명확한 한계 설정(Turn Caps)을 활용하여 코드 품질과 토큰 사용 비용의 균형을 맞추는 것이 핵심입니다.

3. 상세 내용 분해 에이전트 루프(Agentic Loops)의 정의 및 접근 규칙

- 루프의 본질: 에이전트가 작업을 수집하고, 실행하며, 결과를 검증하고, 필요 시 이를 반복한 뒤 최종 응답을 반환하는 주기적 작업 과정입니다.
- 적용 규칙: 모든 작업에 복잡한 루프가 필요한 것은 아니므로, 가장 간단한 접근 방식부터 시작하고 필요한 경우에만 선택적으로 복잡한 패턴을 도입해야 합니다.

4 가지 루프 유형별 상세 분석

- 턴 기반 루프 (Turn-based Loops)
 - 트리거: 사용자의 직접적인 프롬프트 입력.
 - 중단 조건: Claude 가 작업 완료를 판단하거나 추가 컨텍스트가 필요하다고 판단할 때.
 - 적합한 작업: 정기적인 프로세스나 일정이 아닌 단기적인 개별 작업 (예: 버튼 구성요소 생성).
 - 품질 제어 방안: 사람이 개입하는 단계를 SKILL.md 파일로 문서화하여 에이전트가 직접 엔드투엔드로 자가 검증하도록 지원합니다.
 - 검증 스킬 구성 예시: 웹 개발 시 개발 서버를 실행하고 브라우저를 열어 변경 사항을 직접 클릭 확인하며, 브라우저 콘솔의 오류가 0 개인지 확인하고, Chrome DevTools MCP 를 통해 성능 추적 및 Core Web Vitals 를 검사하는 절차를 명시합니다.
- 목표 기반 루프 (Goal-based Loops: /goal)
 - 트리거: 실시간 수동 프롬프트.
 - 중단 조건: 설정한 목표 달성 또는 지정된 최대 턴 수(Turn Caps) 도달.
 - 적합한 작업: 출구기준(Exit criteria)을 명확하게 검증할 수 있는 단일/복합 작업.
 - 작동 방식: Claude 가 작업을 일찍 종료하려 할 때 평가자 모델(Evaluator Model)이 성공 조건을 검증하여 목표가 달성될 때까지 재작업을 지시합니다.
 - 명령어 예시: /goal get the homepage Lighthouse score to 90 or above, stop after 5 tries. (Lighthouse 점수 90 점 이상 달성, 최대 5 회 시도 후 중단).
- 시간 기반 루프 (Time-based Loops: /loop, /schedule)
 - 트리거: 지정된 시간 간격 (Interval).
 - 중단 조건: 사용자의 취소 명령 또는 작업 완료 (PR 머지, 큐 비우기 등).
 - 적합한 작업: 반복적인 작업(입력만 변경되는 작업) 또는 외부 시스템 연동 (예: 매일 아침 Slack 메시지 요약, PR 코드 리뷰 및 CI 실패 수정 확인).

- 실행 환경 분리: 로컬 컴퓨터에서 반복 실행 시 `/loop 5m` 형태를 사용하며, 컴퓨터를 꺼도 클라우드에서 지속되도록 하려면 `/schedule` 루틴을 생성합니다.
- 능동형 루프 (**Proactive Loops**)
 - 트리거: 이벤트 또는 일정 기반 (실시간 인간 개입 없음).
 - 중단 조건: 개별 작업의 목표 달성 시 해당 작업 종료 (루틴 자체는 사용자가 끄기 전까지 유지).
 - 적합한 작업: 버그 리포트 처리, 이슈 분류, 시스템 마이그레이션, 의존성 패키지 업그레이드 등 명확히 정의된 연속 작업 흐름.
 - 효율적인 구성 예시: `/schedule` 로 시간별 확인 -> `/goal` 및 `Skills` 로 검증 기준 지정 -> 동적 워크플로우(Dynamic Workflows)로 3 개 대안 파이프라인 병렬 탐색 및 상호 리뷰 -> 오토 모드(Auto mode)로 승인 대기 없이 자동 실행.

코드 품질 유지 전략 (**Maintaining Code Quality**)

- 깨끗한 코드베이스 유지: Claude 는 기존 코드베이스에 존재하는 패턴과 규칙을 그대로 학습하여 따르므로 코드 베이스의 청결 유지가 우선입니다.
- 자가 검증 수단 제공: 팀이 요구하는 정량적 표준을 `Skills` 로 암호화하여 제공합니다.
- 최신 문서 제공: 사용하는 프레임워크 및 라이브러리의 최신 모범 사례 문서를 에이전트가 쉽게 접근할 수 있도록 연결합니다.
- 제 2 의 에이전트를 통한 코드 리뷰: 작업 에이전트의 편향을 줄이기 위해 독립된 컨텍스트를 가진 별도의 리뷰어 에이전트를 활용합니다 (`/code-review` 스킬 또는 GitHub 용 Code Review 활용).

토큰 사용량 및 비용 관리 방안 (**Managing Token Usage**)

- 적절한 모델 선택: 소규모 작업에는 작은 모델이나 저렴하고 빠른 모델을 할당하고, 고난도 판단에만 성능이 뛰어난 모델을 배치합니다.
- 명확한 완료 기준 설정: 세부적인 성공 조건을 정의하여 불필요한 탐색 사이클을 줄입니다.
- 대규모 작업 전 파일럿 테스트: 수백 개의 에이전트를 생성할 수 있는 동적 워크플로우 실행 전, 전체 작업의 일부에 대해서만 사전 사용량을 측정합니다.
- 정형화된 작업의 스크립트화: 모델이 추론을 반복하게 하는 대신, 스크립트를 실행하도록 전환하는 것이 훨씬 저렴합니다 (예: PDF 폼 채우기 스크립트 실행).

- 모니터링 명령어 활용: /usage (스킬/서브에이전트/MCP 별 토큰 분석), /goal (시도 토큰 수 및 누적 토큰 표시), /workflows (에이전트별 토큰 사용량 확인 및 중단 기능)를 적극 활용합니다.

4. 주요 시사점 및 결론

- 에이전트 관리의 핵심 변화: 단순 프롬프팅에서 에이전트의 트리거, 검증 스킬, 종료 조건을 설계하는 '루프 엔지니어링(Loop Engineering)'으로 패러다임이 전환되고 있습니다.
- 단계적 도입 방법론: 본인이 작업 병목이 되는 영역 중 목표가 명확하거나 주기가 일정한 작업 1 개를 선정하여 자가 검증 스킬(SKILL.md)을 도입하는 것부터 시작해야 합니다.
- 시스템 품질 제어: 개별 실행 결과에 오류가 발생했을 때 단순히 코드를 수정하는 데 그치지 않고, 시스템(Skills 및 평가 조건) 자체에 원인을 반영하여 향후 모든 반복 사이클의 품질을 향상시켜야 합니다.

3.

<https://claude.com/blog/steering-claude-code-skills-hooks-rules-subagents-and-more>

1. 문서 개요

- 웹페이지 한 줄 핵심 정의: 앤트로픽(Anthropic)이 제시하는 Claude Code 제어 프레임워크로, CLAUDE.md, Rules, Skills, Subagents, Hooks, Output styles, Appending the system prompt 등 7 가지 행동 제어 방식을 토큰 비용과 실행 권한 관점에서 비교 및 안내하는 지침서입니다.
- 작성 목적 및 주요 대상: 개발자가 Claude Code 를 작업 방식에 맞춰 커스텀할 때, 맥락 수용량(Context Cost), 압축(Compaction) 동작, 실행 권한을 고려하여 최적의 제어 방식을 선택하고 가동할 수 있도록 돕기 위함입니다. 주요 대상은 Claude Code 를 도입하여 소프트웨어를 구축하는 개발자 및 AI 엔지니어링 리더입니다.

2. 핵심 주제 및 요약

- Claude Code 의 행동을 제어하는 방식은 로드 시점, 압축 유지 방식, 컨텍스트 비용, 제어 권한에 따라 7 가지 형태로 구별됩니다.
- 시스템 전반에 상시 적용해야 하는 최상위 지침은 CLAUDE.md 와 Rules 로 관리하고, 절차적 작업은 Skills, 독립적 작업은 Subagents, 확정적 자동화는 Hooks 로 분리하는 것이 컨텍스트 보존과 제어 품질 측면에서 핵심입니다.
- 단순 프롬프트 입력에 그치지 않고 개발 환경의 복잡성에 따라 7 가지 도구를 적절히 조합하여 사용하는 구조적 제어 체계가 필요합니다.

3. 상세 내용 분해 7 가지 행동

제어 방식 비교 개요

개발자는 작업 환경의 복잡성과 컨텍스트 사용량(Token Cost)을 고려하여 아래의 7 가지 방식을 선택적으로 활용할 수 있습니다.

- **CLAUDE.md (루트):** 세션 시작 시 로드되며 전체 세션 동안 맥락에 유지됩니다. 빌드 명령, 디렉토리 구조, 코딩 컨벤션을 전달하는 데 적합하며, 컨텍스트 사용 비용이 매우 높습니다.
- **CLAUDE.md (하위 디렉토리):** 해당 하위 디렉토리 내의 파일을 작업할 때 주문형(On-demand)으로 로드되어 특정 디렉토리 전용 규약을 적용하는 데 적합합니다.
- **Rules (.claude/rules/):** 세션 시작 시(사용자 수준) 또는 특정 파일 경로가 일치할 때(Path-scoped) 로드됩니다. Zod 입력 검증과 같은 하드 제약 조건이나 규약 설정에 사용됩니다.
- **Skills (.claude/skills/):** 이름과 설명은 세션 시작 시 로드되지만 본문은 슬래시 명령(/)이나 에이전트 매칭을 통해 호출될 때만 로드됩니다. 배포 절차, 체크리스트 등 절차적 워크플로우에 적합합니다.
- **Subagents (.claude/agents/):** 별도의 독립된 맥락 창에서 작업을 수행하며, 최상위 메인 세션에는 최종 결과 요약과 메타데이터만 반환합니다. 심층 검색, 로그 분석 등 독립된 격리 작업에 활용됩니다.
- **Hooks:** 파일 수정, 툴 호출, 세션 시작 등 특정 이벤트 시점에 자동으로 실행됩니다. 린터 실행, 배포 후 알림, 커맨드 차단 등 확정적 자동화에 적합합니다.
- **Output styles & System Prompt Appends:** 시스템 프롬프트 수준을 수정하여 Claude 의 역할, 어조, 출력을 전역적으로 변경할 때 사용됩니다.

CLAUDE.md 파일 구조화 및 운영 규칙

- 기본 특징: 프로젝트 루트에 위치하는 마크다운 파일로, 세션 시작 시 로드되어 상시 유지됩니다. 대화 내용이 요약(Compaction)되더라도 다시 불러옵니다.
- 루트와 하위 디렉토리 분리:
 - 루트 CLAUDE.md: 프로젝트 전체 빌드 명령, 디렉토리 레이아웃, 모노레포 구조 등 필수 상시 정보만 최소한으로 작성합니다.
 - 하위 디렉토리 CLAUDE.md: 특정 폴더 하위 파일(예: app/api/)을 읽을 때만 해당 디렉토리의 설정이 로드되어 토큰 소모를 방지합니다.
- 운영 팁: 파일 분량이 커지면 토큰이 낭비되고 지침 준수율이 떨어지므로 200 줄 이하로 유지해야 합니다. 팀별 개별 지침은 Path-scoped Rules 나 Skills 로 이관하여 관리하는 것이 권장됩니다.

규칙(Rules)을 통한 정밀 제약 설정

- 파일 위치: .claude/rules/ 디렉토리에 마크다운 형식으로 작성합니다.
- 경로 범주 지정(**Path-scoped Rules**): YAML 헤더에 paths 항목을 지정하면 지정된 파일에 접근할 때만 지침이 로드됩니다.
- 적용 예시:

```

---
paths:
- "src/api/**"
- "**/*.handler.ts"
---
```

모든 API 핸들러는 처리 전에 Zod 를 사용하여 입력을 검증해야 합니다.

- 활용 지침: 프로젝트 전체가 아닌 특정 디렉토리나 파일 그룹에만 적용되어야 하는 명확한 제약 조건은 Rules 를 이용해 처리하는 것이 효과적입니다.

스킬(Skills)을 이용한 절차적 워크플로우 재사용

- 구성 요소: .claude/skills/ 폴더 내의 SKILL.md 파일을 통해 이름, 설명, 실행 지침, 스크립트, 리소스를 정의합니다.
- 작동 방식: 세션 시작 시에는 이름과 설명만 상주하므로 컨텍스트 소모가 적습니다. 필요시 /code-review 와 같은 명령으로 호출하면 전체 가이드라인 본문이 로드되어 작업을 수행합니다.

- 압축 공유 예산: 세션이 길어져 압축이 발생하면 호출된 스킬들은 지정된 예산 한도 내에서 최신 항목 순으로 재주입되며, 오래된 스킬 정보는 제거될 수 있습니다.

서브에이전트(Subagents)를 통한 작업 격리

- 파일 위치: `.claude/agents/` 디렉토리에 작성하며, 시스템 프롬프트 형태의 구조를 갖습니다.
- 컨텍스트 격리 효과: 서브에이전트는 독립된 맥락 창(Context Window)에서 별도로 작동합니다. 실행 과정에서 발생하는 수많은 중간 데이터는 메인 세션에 쌓이지 않고, 오직 최종 결과 메시지와 메타데이터만 메인 세션으로 전달됩니다.
- 선택 기준: 실행 과정을 메인 창에서 실시간으로 확인하며 제어하고 싶다면 Skills 를 사용하고, 중간 과정이 메인 컨텍스트를 더럽히지 않기를 바란다면 Subagents 를 선택합니다.

훅(Hooks)을 활용한 확정적 자동화

- 정의 및 특징: lifecycle 이벤트(파일 수정, 툴 실행, 세션 시작/종료 등)에 반응하여 자동으로 트리거되는 사용자 정의 명령어, HTTP 엔드포인트, LLM 프롬프트입니다.
- 유형:
 - 확정적 실행: `command`, `http`, `mcp_tool` (설정 파일 기반으로 규칙에 따라 확정 동작)
 - 판단 기반 실행: `prompt`, `agent` (LLM 판단을 거쳐 동작 수행)
- 주요 활용 사례: 코드 수정 직후 자동 린터 실행, 보안상 위험한 명령어 실행 차단, 완료 시 외부 서비스(Slack 등)로 결과 통보.

4. 주요 시사점 및 결론

- 컨텍스트와 권한의 트레이드오프 이해: 상시 로드 방식(CLAUDE.md)은 높은 제어력을 제공하지만 높은 토큰 비용이 발생하며, 온디맨드 방식(Skills, Subagents, Hooks)은 맥락을 보호하면서 효율적인 제어를 가능하게 합니다.
- 구조적 분리를 통한 품질 향상: 단일 CLAUDE.md 파일에 모든 지침을 몰아넣기보다, 특성에 따라 Rules, Skills, Subagents, Hooks 로 분산 설계해야 맥락의 오염을 막고 지침 이행률을 높일 수 있습니다.
- 체계적 에이전트 운용: Claude Code 를 도입할 때는 개발 프로세스 내에서 결정론적으로 자동화할 영역(Hooks)과 LLM 추론이 필요한 영역(Skills/Subagents)을 명확히 구분하여 거버넌스를 구축해야 합니다.